Relatório Final — Projeto Marketplace (Etapa 2)

Introdução

Este relatório consolida o projeto Marketplace (banco `marketplace_db`, MySQL 8) da disciplina Laboratório de Banco de Dados 2026/2 (Grupo 9) Etapa 2 (operação, interface e desempenho), conforme o item B8 do enunciado. O sistema modela uma plataforma em que usuários vendem e compram produtos, com carrinho, pedidos e histórico de status.

Os scripts executáveis ficam em `sql/` (`01_ddl` a `08_view_perfil`) e os documentos de apoio em `docs/`.

Parte II — Etapa 2: operação, interface e desempenho

A Etapa 2 pôs o banco em uso: uma aplicação Spring Boot (Java, MySQL 8) exerce o esquema, e os scripts 04 a 08 cobrem índices, senhas, auditoria, transações e acesso. O SGBD é o mesmo da Etapa 1 e o DDL é o `01_ddl.sql`; o Hibernate roda com `ddl-auto=none`, então nenhuma tabela é gerada a partir das classes.

1. Aplicação com interface (B1)

A aplicação é uma API REST com interface web em uma página (Spring serve o `index.html`), conectada ao `marketplace_db`.

Entidade	Operações expostas
Usuário (+ cliente, vendedor)	Criar (POST `/users`, `/users/{id}/client`, `/users/{id}/seller`), login (POST `/login`)
Produto	Criar, listar e buscar por nome, consultar, alterar (PUT), remover (DELETE)
Categoria	CRUD em `/categories`
Fornecedor	CRUD em `/suppliers` e vínculo produto–fornecedor em `/products/{id}/suppliers`
Carrinho	Consultar, incluir e remover item
Pedido	Checkout, consultar, listar por cliente e alterar status (PATCH `/orders/{id}/status`)

Operação transacional central — checkout (`OrderService.checkout`, anotado com `@Transactional`). Em uma única transação ela baixa o estoque de cada produto, grava o pedido (`tb_order`), seus itens (`tb_orderItem`) e o primeiro status "aberto" (`tb_order_status`), e esvazia o carrinho (`tb_buyItem`, `tb_cart`). Se qualquer item não tiver estoque, a exceção desfaz tudo (item 3).

Tratamento de erro visível. Um `GlobalExceptionHandler` converte as falhas em mensagens JSON legíveis, nunca a exceção bruta do driver: violação de regra de negócio → HTTP 422 com `{"error": "Estoque insuficiente para …"}`; recurso inexistente → 404; validação de campos → 400 com o

campo e a mensagem. Transições de status inválidas (por exemplo, ENTREGUE → PAGO) retornam "Não é possível ir de X para Y".

2. Camada de acesso a dados e SQL injection (B2)

Todo acesso ao banco passa por Spring Data JPA. As buscas usam métodos derivados (`findByNameContainingIgnoreCase`) e as alterações de estoque usam `@Query` com `@Param` (`decreaseStock`, `increaseStock`), que o driver envia como parâmetros, nunca concatenados. O `decreaseStock` ainda leva a condição `quantity >= :qty` na própria UPDATE, o que impede estoque negativo mesmo com requisições simultâneas.

Tentativa de injeção e resultado. No endpoint `/products` o grupo buscou primeiro "Samsung" (2 produtos) e depois `' OR '1'='1`. A aplicação devolveu lista vazia `[]`, tratando a entrada como texto literal, enquanto a tabela tinha 56 produtos no momento do teste. Um teste isolado não prova a ausência de vulnerabilidades em outros pontos.

Credenciais. O `application.properties` passou a ler `DB_USERNAME` e `DB_PASSWORD` de variáveis de ambiente.

3. Transações e controle de concorrência (B3)

Os testes estão em `sql/07_transacoes_concorrencia.sql`, sobre o produto 1 (estoque inicial de 61 unidades).

Falha provocada no checkout. A regra "estoque insuficiente" lança `BusinessException` no meio da transação, depois de o pedido já ter sido montado e de itens anteriores terem tido o estoque baixado. O `@Transactional` desfaz tudo (rollback) e o banco fica como estava.

Nível de isolamento. O InnoDB usa REPEATABLE READ por padrão, e o grupo não o alterou. Proposta de justificativa: o checkout lê o carrinho e depois altera o estoque, e o REPEATABLE READ mantém uma leitura consistente dentro da transação, enquanto a `UPDATE ... WHERE quantity >= :qty` serializa as baixas do mesmo produto por bloqueio de linha. READ COMMITTED reduziria bloqueios de leitura, mas não há contenção que o justifique neste volume.

Experimento	Comandos	Resultado observado
ROLLBACK	`START TRANSACTION; UPDATE tb_product SET quantity = quantity - 5 WHERE id_product = 1; ROLLBACK;`	Estoque foi a 56 dentro da transação e voltou a 61 após o ROLLBACK
COMMIT	O mesmo UPDATE, seguido de `COMMIT`	Estoque ficou em 56 de forma permanente; depois restaurado a 61 com outra transação
Concorrência	Sessão 1: `START TRANSACTION; SELECT … WHERE id_product = 1 FOR UPDATE;` Sessão 2: o mesmo `SELECT … FOR UPDATE`	A sessão 2 esperou 5 s e falhou com o erro 1205 (Lock wait timeout exceeded); após o ROLLBACK da sessão 1 o bloqueio foi liberado

4. Otimização de consultas (B4)

O estudo (`sql/04_indices_explain.sql`) parte da busca de produto por nome, a mesma que a aplicação executa em `/products`. Consulta analisada:

```
EXPLAIN SELECT * FROM tb_product WHERE name = 'Celular Samsung Galaxy A54';
CREATE INDEX idx_product_name ON tb_product (name);
```

Métrica do EXPLAIN	Antes do índice	Depois do índice
Tipo de acesso (`type`)	ALL (varredura completa)	ref
Índice usado (`key`)	NULL	`idx_product_name`
Linhas estimadas (`rows`)	56	1

Leitura do plano. Sem índice o MySQL lê a tabela inteira e filtra linha a linha. Com o índice em name ele localiza a entrada na árvore B+ e lê só a linha correspondente: a estimativa caiu de 56 para 1 (cerca de 98%). Uma coluna só de igualdade não tem ordem de colunas a discutir. Esse índice não ajuda em `LIKE '%texto%'`, que a busca `ContainingIgnoreCase` da aplicação usa; ali o MySQL continua varrendo.

Limite honesto da medição. `rows` é uma estimativa do otimizador, não um tempo. Com 56 produtos a diferença de tempo é imperceptível, e o ganho real só aparece em volume maior.

5. Segurança e controle de acesso (B5)

Perfis (`sql/08_view_perfil.sql`). Três papéis (roles) do MySQL, com o princípio do menor privilégio:

Perfil (usuário)	Pode	Não pode
`role_admin` (`app_admin`)	`SELECT`, `INSERT`, `UPDATE`, `DELETE` em todo o `marketplace_db`	DDL (criar ou apagar tabelas)
`role_vendedor` (`app_vendedor`)	Ler categorias e fornecedores; `SELECT`, `INSERT`, `UPDATE` em produtos; CRUD em `tb_product_supplier`; ler `vw_user_public`	`DELETE` em `tb_product` (`REVOKE`); ver senhas, pedidos e clientes
`role_analista` (`app_analista`)	Ler `vw_sales_by_category` e `vw_user_public`	Qualquer escrita e qualquer tabela base

Visões de restrição. `vw_user_public` expõe só `id_user`, `user` e `email`, escondendo `password`. `vw_sales_by_category` entrega pedidos, unidades e receita por categoria, sem pedidos cancelados, sem dar acesso às tabelas de pedido. Os usuários `app_*` são criados com senha de marcador (`TROQUE_SENHA_*`), que deve ser trocada por segredo real fora do repositório.

Senhas da aplicação. A coluna `tb_user.password` guarda hash BCrypt com sal (`$2a$10$…`): o script `05_senhas_bcrypt.sql` troca as 45 senhas de exemplo da carga, e a aplicação valida o login com `BCryptPasswordEncoder.matches()`. Cada hash leva um sal diferente embutido no próprio texto, então duas senhas iguais não geram o mesmo hash. Falha de login devolve a mesma mensagem (“E-mail ou senha inválidos”) para e-mail inexistente e senha errada.

Dados pessoais e cuidados.

Dado	Onde	Cuidado adotado
E-mail e nome de usuário	`tb_user`	Expostos a perfis restritos só por visão; e-mail único; carga com dados fictícios
Senha	`tb_user.password`	Hash BCrypt com sal; fora de `vw_user_public`
Telefone	`tb_client`, `tb_supplier`	Opcional; sem acesso para `role_vendedor` e `role_analista` ao cliente
CNPJ	`tb_seller`, `tb_supplier`	Dado de pessoa jurídica, tratado como cadastral; carga fictícia
Histórico de compras	`tb_order*`	Sem acesso para o perfil vendedor; analista só vê agregado por categoria

6. Gatilhos e auditoria (B6)

A tabela `tb_audit_product` (`sql/06_auditoria.sql`) guarda `id_product`, `usuario` (`USER()`, a conta conectada ao banco), `data_alteracao` (`DEFAULT CURRENT_TIMESTAMP`), `operacao`, `valor_antigo` e `valor_novo`. Dois gatilhos AFTER a alimentam.

Gatilho	Evento	O que registra
`trg_audit_product_update`	AFTER UPDATE em `tb_product`	Só quando `OLD.value <> NEW.value`: preço antigo e novo
`trg_audit_product_insert`	AFTER INSERT em `tb_product`	Preço de cadastro; `valor_antigo` fica NULL

Por que auditar preço e cadastro. O preço é o dado de negócio mais sensível do catálogo: ele define o que o cliente paga (RN09 copia o preço vigente para o carrinho) e uma alteração indevida tem efeito financeiro direto. O pedido já congela o preço em `tb_orderItem.unit_value`, então o histórico de vendas não muda, mas ninguém sabia quem alterou o preço do catálogo e quando. O gatilho de UPDATE ignora mudanças de estoque, que a aplicação faz a cada venda e gerariam ruído.

Testes. O preço do "Celular Samsung Galaxy" foi alterado de R$ 1.949,27 para R$ 1.999,27 e a auditoria registrou os dois valores, o usuário e a data. O produto "Mouse Gamer Teste" (R$ 150,00) gerou um registro INSERT com valor anterior NULL.

Limitações. `USER()` identifica a conta do banco (a conta da aplicação), não a pessoa logada na interface. Os testes alteraram dados da carga (um preço e um produto novo, que provavelmente explica as 56 linhas vistas em B2 e B4 contra 55 na carga original); devem ser revertidos antes da demonstração.

7. Projeto físico e armazenamento (B7)

Revisão dos tipos. O grupo revisou os tipos à luz do custo de armazenamento e da evolução do modelo e decidiu não alterar nenhum tipo.

Estimativa de crescimento (2 anos). Premissas hipotéticas: 500 pedidos por dia, 2,5 itens por pedido e 1,8 registros de status por pedido, em 730 dias.

Tabela	Cálculo	Linhas em 2 anos	Espaço estimado (dados + índices)
`tb_order`	500 × 730	365.000	~35 MB
`tb_orderItem`	365.000 × 2,5	912.500	~55 MB
`tb_order_status`	365.000 × 1,8	657.000	~22 MB
`tb_audit_product`	20 × 730	14.600	~1 MB
Total		**1.949.100**	**~115 MB**

Com crescimento de 3× (1.500 pedidos/dia) o banco chega a ~5,8 milhões de linhas e ~350 MB; com 10× (5.000 pedidos/dia), a ~19,3 milhões de linhas e ~1,2 GB, ainda confortável para um servidor comum. O volume não é o problema: `tb_order_status` e `tb_orderItem` são tabelas de apenas inserção e crescem sem parar, e carrinhos abandonados também acumulam.

Plano de backup.

Decisão	Proposta
Metas	RPO de até 15 minutos (logs binários) e RTO de até 2 horas
O que copiar	Banco `marketplace_db` completo
Como	`mysqldump --single-transaction --routines --triggers --events`, consistente e sem bloquear vendas (tabelas InnoDB); logs binários ativos (`log_bin=ON`, `binlog_format=ROW`) para recuperação até um instante; acima de ~10 GB, backup físico (Percona XtraBackup)
Frequência	Completo diário às 03h; cópia dos logs binários a cada 15 minutos
Retenção	7 diários, 4 semanais e 6 mensais; logs binários por 7 dias
Onde	Regra 3-2-1: 3 cópias, 2 mídias, 1 fora do local; dump criptografado, pois contém dados pessoais e hashes
Teste de restauração	Mensal, em `marketplace_db_teste`: restaurar, comparar contagem de linhas por tabela, rodar as 15 consultas de `03_consultas.sql` e registrar o tempo para validar o RTO

Particionamento e arquivamento. Não se propõe particionar: com ~1,9 milhão de linhas em dois anos os índices resolvem as consultas, e o InnoDB não aceita chaves estrangeiras em tabelas particionadas, o que quebraria `fk_orderitem_order`, `fk_orderstatus_order` e `fk_order_client`. Para carrinhos abandonados, a política é arquivar ou apagar os antigos.

Parte IV — Declaração de uso de inteligência artificial

O grupo usou um assistente de IA como apoio ao estudo, à organização do trabalho e à criação de código:

- **Organização de documentos:** estruturar o que cada artefato exigia e cruzar o enunciado com o repositório, para ver o que estava pronto e o que faltava.
- **Formatação:** melhorar a apresentação dos documentos.
- **Guia em conteúdos novos:** explicar os temas e orientar.

- **Sistema:** ajudar a guiar o trabalho e a perceber erros.
- **SQL:** revisar DDL, carga, consultas, visões, perfis de acesso e gatilhos, e apontar inconsistências entre os scripts e as regras de negócio.
- **Testes:** apoio na escrita de testes JUnit, Postman, Mockito e Playwright.

Como o resultado foi verificado. Nada gerado pela IA entrou no projeto sem ser lido e testado pela equipe. Os scripts foram executados em base limpa, e os erros da própria IA foram corrigidos com a equipe procurando entender os motivos. Os dados da carga são fictícios, e a IA não recebeu credenciais nem dados pessoais reais.

Conclusão

O banco entrega 12 relações normalizadas até a FNBC, 20 regras rastreáveis, índice, auditoria, perfis de acesso, senhas com hash e plano físico, e a aplicação executa o checkout de forma transacional com erros legíveis.